ESP32 Bluetooth MP3 Player

Technical Documentation & Developer Guide

Version	1.0.0
Platform	ESP32 (ESP-IDF v5.5.2)
Author	M Praveen Kumar
Date	March 2026
Status	Production Ready

1. Project Overview

The ESP32 Bluetooth MP3 Player is an embedded system that downloads an MP3 audio file over WiFi, stores it on an SD card, decodes it in real-time using the libmad fixed-point decoder, and streams the resulting PCM audio to a Bluetooth A2DP speaker — all on a single ESP32 microcontroller.

The system is designed for offline, autonomous playback: after the initial one-time WiFi download, it operates entirely without a network connection, looping the audio file continuously over Bluetooth.

1.1 Key Features

- One-time WiFi download with persistent NVS flag — no re-download on reboot
- Real-time MP3 decoding via libmad fixed-point library
- Bluetooth A2DP source — streams audio to any A2DP-compatible speaker
- SD card storage via SPI interface
- Automatic device discovery and reconnection
- Continuous looped playback
- Pre-buffering to eliminate startup glitches

1.2 System Architecture

The pipeline follows a clean producer-consumer pattern:

Data Flow: SD Card (MP3) → libmad Decoder → FreeRTOS Ring Buffer → A2DP Audio Callback → SBC Encoder → Bluetooth RF → Speaker

2. Hardware Requirements

2.1 Components

Component	Specification	Notes
Microcontroller	ESP32 (any variant)	Classic BT required — not ESP32-S2/C3
SD Card Module	SPI-based	Tested with SDHC cards
SD Card	Any size, FAT32	Tested: 15 GB SDHC
Bluetooth Speaker	A2DP sink device	Tested: Saregama Carvaan
Power Supply	3.3V / 5V USB	Standard ESP32 dev board USB

2.2 Pin Connections

Signal	ESP32 GPIO	SD Card Pin
MOSI (Data Out)	GPIO 22	DI
MISO (Data In)	GPIO 21	DO
CLK (Clock)	GPIO 26	CLK
CS (Chip Select)	GPIO 16	CS
3.3V	3.3V	VCC
GND	GND	GND

3. Software Architecture

3.1 Boot Phases

The firmware operates in two mutually exclusive phases, determined by an NVS flag at startup:

Phase	Trigger	Actions	Outcome
Phase 1 — Download	NVS flag not set	Connect WiFi, HTTP download, save to SD	Reboot into Phase 2
Phase 2 — Playback	NVS flag set	Open MP3, create ring buffer, start BT	Continuous audio streaming

Design Note: WiFi and Bluetooth never run simultaneously. This avoids radio contention on the ESP32's shared 2.4 GHz antenna and frees ~30 KB of heap for the BT stack.

3.2 FreeRTOS Tasks

Task	Function	Stack	Priority	Lifecycle
mp3_decoder_task	Reads MP3 from SD, decodes via libmad, writes PCM to ring buffer	12 KB	5	Created on A2DP connect
prebuffer_then_start_task	Waits for ring buffer to fill, then triggers A2DP START	1.5 KB	4	Created on A2DP connect, self-deletes
Main Task (app_main)	Initialization, BT start, idle loop	Default	1	Runs forever

3.3 Ring Buffer Pipeline

A FreeRTOS BYTEBUF-type ring buffer of 8 KB serves as the decoupling layer between the decoder and the Bluetooth audio callback:

- Decoder writes decoded PCM frames (~4608 bytes per frame at 44.1 kHz stereo)
- Audio callback reads exactly the number of bytes requested by the BT stack
- If the buffer is empty, the callback fills the remainder with silence (zeros)
- The callback always returns the full requested length to keep the BT stack stable

4. Key Modules

4.1 ID3 Tag Handling

MP3 files commonly begin with an ID3v2 metadata block containing artist, album, and cover art data. libmad cannot decode this and raises MAD_ERROR_LOSTSYNC if fed raw ID3 bytes.

The skip_id3_tag() function reads the first 10 bytes, checks for the 'ID3' magic string, decodes the synchsafe integer size field (4 x 7-bit bytes), and seeks the file pointer past the tag before any decoding begins.

4.2 libmad Decoder Loop

libmad is a fixed-point MP3 decoder. The decode loop per iteration:

1. Read up to 2 KB from SD card into input buffer
2. Feed buffer to mad_stream_buffer()
3. Decode one MP3 frame with mad_frame_decode() — produces ~1152 samples
4. Synthesize PCM with mad_synth_frame()
5. Convert 28-bit fixed-point samples to int16_t via bit-shift
6. Write stereo int16 PCM to ring buffer (blocking with 100 ms timeout)
7. On EOF: seek to file start and loop

4.3 Bluetooth A2DP Flow

Event	Handler Action
CONNECTION_STATE_EVT (Connected)	Start mp3_decoder_task and prebuffer_then_start_task
AUDIO_CFG_EVT	Log SBC codec negotiation (type 0 = SBC)
MEDIA_CTRL_ACK_EVT (START)	Set bt_streaming = true
CONNECTION_STATE_EVT (Disconnected)	Reset flags, restart GAP discovery

5. Configuration Reference

5.1 Build-Time Defines

Define	Default	Description
WIFI_SSID	"YOUR_SSID"	WiFi network name for download phase
WIFI_PASSWORD	"YOUR_PASSWORD"	WiFi password
MP3_URL	http://<server-ip>:8080/MUSIC.MP3	HTTP server URL for MP3 download
MP3_PATH	"/sdcard/MUSIC.MP3"	Path on SD card where file is stored
BT_TARGET_DEVICE	"Carvaan"	Substring to match in discovered BT device names
PCM_RING_BUF_SIZE	8192 (8 KB)	FreeRTOS ring buffer size in bytes
PCM_PREBUFFER_BYTES	4096 (4 KB)	Minimum fill before A2DP START is sent
MP3_READ_BUF_SIZE	2048 (2 KB)	Chunk size for SD card reads

5.2 Supported MP3 Formats

Property	Recommended	Notes
Sample Rate	44100 Hz	SBC codec negotiated at 44.1 kHz; other rates may sound wrong
Channels	Stereo	Mono supported — duplicated to both channels
Bitrate	128 kbps CBR	320 kbps may cause decoder underruns on tight heap
Encoding	CBR	VBR may cause sync issues with libmad
File Size	Up to SD capacity	Tested: 3 MB file on 15 GB SDHC card
Tags	ID3v2 (any)	ID3 tags automatically detected and skipped

6. Setup & Deployment Guide

6.1 Prerequisites

- ESP-IDF v5.5.2 installed and configured
- libmad component added to project components/ folder
- SD card formatted as FAT32
- HTTP server running on local network (e.g. Python: python -m http.server 8080)
- Target Bluetooth speaker powered on and in pairing mode

6.2 First-Time Flash

8. Update WIFI_SSID, WIFI_PASSWORD, and MP3_URL defines in main.c
9. Set BT_TARGET_DEVICE to match your speaker's advertised name
10. Build and flash:

```
idf.py fullclean && idf.py build && idf.py flash monitor
```

11. ESP32 connects to WiFi, downloads MP3 to SD card, reboots automatically
12. After reboot: ESP32 scans for target BT device, connects, and begins streaming

6.3 Changing the MP3 File

13. Replace the file on your HTTP server
14. Erase flash to clear the NVS flag:

```
idf.py erase-flash && idf.py flash monitor
```

15. ESP32 re-downloads the new file automatically

6.4 Changing the Target Speaker

Edit the BT_TARGET_DEVICE define at the top of main.c:

```
#define BT_TARGET_DEVICE    "Mivi"    // or "JBL", "Sony", etc.
```

Reflash with idf.py flash monitor. No erase needed — WiFi/download phase is skipped.

7. Troubleshooting

Symptom	Cause	Fix
Ring buffer always 0	Decoder task crashing silently	Check heap > 50 KB before BT init; increase decoder stack
MAD_ERROR_LOSTSYNC (0x0031)	ID3 tag not skipped	Ensure skip_id3_tag() is called after fopen()
btc_media_aa_prep_sbc_2_send no buffer	BT OSI heap exhausted	Increase BR/EDR ACL connections in menuconfig; reduce ring buffer size
Device not found in discovery	Speaker off or wrong name	Power cycle speaker; check printed device names match BT_TARGET_DEVICE
Download fails	Wrong IP or server not running	Run python -m http.server 8080 in MP3 folder; verify IP
SD mount failed	Wrong pins or card not formatted	Verify GPIO pins match defines; format card as FAT32
Audio glitches / stuttering	Buffer underruns	Increase PCM_PREBUFFER_BYTES; reduce MP3 bitrate to 128 kbps CBR

8. Memory Profile

Component	Heap Used (approx.)	Notes
BT Controller + Bluedroid	~65 KB	After BLE memory release
libmad structs (malloc)	~25 KB	stream + frame + synth
PCM output buffer	~4.5 KB	1152 samples x 2 ch x 2 bytes
MP3 input buffer	2 KB	MP3_READ_BUF_SIZE
FreeRTOS ring buffer	8 KB	PCM_RING_BUF_SIZE
Decoder task stack	12 KB	Heap-allocated by FreeRTOS
Pre-buffer task stack	1.5 KB	Self-deletes after START
Free at streaming start	~33 KB	Sufficient for stable operation

9. Project Author

Field	Details

Field	Details
Name	M Praveen Kumar
Role	Embedded Systems Developer
Project	ESP32 Bluetooth MP3 Player
Platform	ESP32 / ESP-IDF v5.5.2
Year	2026

10. Repository

GitHub: https://github.com/Mpraveenkumar61/esp32-bluetooth-mp3-player

10.1 Project Structure

```
esp32-bluetooth-mp3-player/
  main/
    main.c          — Full application source
    CMakeLists.txt  — Component build config
  components/
    libmad/         — Fixed-point MP3 decoder library
  CMakeLists.txt    — Top-level build file
  sdkconfig         — ESP-IDF menuconfig settings
  .gitignore        — Excludes build/, *.bin, *.elf
  README.md         — Quick-start guide
```

10.2 Build Environment

Tool	Version
ESP-IDF	v5.5.2
Compiler	xtensa-esp32-elf-gcc
Target Chip	ESP32 (v3.1)
Flash Size	4 MB
CPU Frequency	160 MHz

11. Future Enhancements

- **Multiple MP3 files:** Playlist support

- **Skip / pause via GPIO:** Button input
- **Show track name, progress:** OLED display
- **Software gain adjustment in decoder:** Volume control
- **Update MP3 over-the-air without USB:** OTA download
- **Browser-based track selector via ESP32 WiFi AP mode:** Web UI